Ders 13 Genişletilmiş Notu: İteratif Algoritmaların Analizi: Toplam Formülleri, Geometrik Seri, Rastgele Sayılara Giriş ve Lab-2

Ders 13 Genişletilmiş Notu: İteratif Algoritmaların Analizi: Toplam Formülleri, Geometrik Seri, Rastgele Sayılara Giriş ve Lab-2

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Ders Başında Bir Kod İncelemesi: Polinom Değeri
- 2. T'nin Değeri ile Karmaşıklık Farklı Şeylerdir
- 3. İç İçe Döngü: İç Sınırı i^2 Olan Döngü
- 4. `while` ile Katlanarak Artan Döngü
- 5. Geometrik Seri Formülünün İspatı
- 6. Polinom Toplam, Üstel Toplam ve $\sum i \cdot a^i$
- 7. Bu Döngüler Kaç Kez Döner?
- 8. `if` İçeren Döngü ve `else` Kolu
- 9. Üst Sınırı Önceden Hesaplanan Döngü
- 10. Ara Sorular: Mantık Hatası, Global Değişkenler, Koşulda Atama
- 11. Üç İç İçe Döngü
- 12. Alt Sınırı 3 Olan Döngü ve Karmaşıklık Hesabı
- 13. Rastgele Sayılara Giriş: Çember Üzerinde Desen
- 14. Tekrar Sorunu
- 15. Uğrama Sayıları ve Tek Düze Dağılım
- 16. Gerçek ve Sözde Rastgele Sayı Üreteçleri
- 17. ANSI C Üreteci ve Çekirdek Değer
- 18. `srand(time(NULL))`
- 19. Ders Sonu Soruları: `srand` ve Kütüphane Kodları
- 20. Seyrek Matris Kodunda Hata Arama ve Dev-C++ Debug Kullanımı
- 21. Polinom Koduna Dönüş
- 22. `exchange` Değişkeni ve `do-while`
- 23. Laboratuvar 2: Seyrek Matrisin Sıkıştırılması

1. Ders Başında Bir Kod İncelemesi: Polinom Değeri

Kayıt, laboratuvarın ardından gelen dersle başlar. Hoca konuya geçmeden önce bir öğrencinin ekranını paylaştığı kodu inceledi. Program kullanıcıdan bir polinom alıyor, katsayıları bir matriste tutuyor ve verilen x için polinomun değerini hesaplıyordu. $f(3)$ için 13.00 gibi doğru sonuçlar görünüyor, küçük x değerlerinde sorun çıkmıyordu. Ama $x = 30$ girilince ekranda sayı yerine garip bir ifade beliriyordu. Altyazıda bu ifade "bir çarpı IO" ve "number of IO" olarak geçiyor.

Hoca hesabı yapan fonksiyonu satır satır izledi. Toplam, matrisin `denklem[0][0]` gözündeki değerle başlatılıyor, `tmp` değişkeni x'i tutuyor ve kendisiyle çarpılarak üsleri üretiyordu. Hoca

algoritmada bir yanlış görmedi. `tmp` ile yapılan çarpmada otomatik tip dönüşümü (cast) olacağını, sonrasında toplamın, katsayılar tam sayı olsa bile, `double` olarak hesaplanmaya devam edeceğini söyledi.

Sonra sorunun yazdırma biçiminde olup olmadığına baktı. `%.3f` yerine `%3.3f`, ardından `%3.3lf` denendi, sonuç değişmedi. İkinci dereceden bir polinomda $x = 30$ için beklenen değer yalnızca dokuz yüzlü bir sayı olduğundan, sayının sığmaması da sebep olarak görünmüyordu. Hoca bunu dersin sonuna bıraktı ve hesabın yapıldığı satıra bir breakpoint konup toplamın ara değerlerine bakılması gerektiğini söyledi. Kodun devamı 21. bölümdedir.

2. T'nin Değeri ile Karmaşıklık Farklı Şeylerdir

Hoca derse Fatih Hoca'nın sayfasındaki slaytlardan, iteratif algoritmaların analizinden devam etti. Soruların hepsi aynı kalıptadır: Verilen kod parçası çalıştırıldığında `T`'nin değeri n'e bağlı olarak ne olur? Kodlar artık C olarak değil, MATLAB benzeri bir sözde kodla yazılıyor. Hoca bunların istenirse kolayca C'ye dönüştürülebileceğini söyledi.

Önce iki kavramın karıştırılmaması gerektiğini vurguladı. `T`'nin değerini bulurken toplam veya çarpım sembolünün içine ne yazacağımıza dikkat ederiz. Algoritmanın karmaşıklığı ise başka bir hesaptır: Temel işlemin kaç kez yapıldığını sayar. Örneğin bir döngüde `T = T + 1` yerine `T = T + T` yazılmış olsun. Karmaşıklık açısından değişen bir şey yoktur, döngü yine n kez döner ve n kez toplama yapılır. Ama `T` her turda iki katına çıktığı için değeri n tane ikinin çarpımına, yani 2^n'e gider.

> **Hocanın vurgusu:** `T`'nin değeriyle algoritmanın karmaşıklığını birbirine karıştırmayın. Bu ayrım bazen unutuluyor.
>
> **Ders sonrası not:** 2^n sonucu `T`'nin 1 ile başlatıldığı durumda geçerlidir (python ile doğrulandı). `T = 0` ile başlayan bir döngüde `T + T` hep 0 kalır. Slayttaki bu örneğin başlangıç değeri büyük olasılıkla 1'dir.

3. İç İçe Döngü: İç Sınırı i^2 Olan Döngü

Slayttaki kod şudur:

```
T = 0
for i = 1:N
    for j = 1:i*i
        T = T + 1
    end
end
```

Hocanın yöntemi her soruda aynıdır: Önce toplam sembollerini sınırlarıyla yazarız, sonra içeriden dışarı doğru çözeriz.

3.1 `T = T + 1`

Dış döngü $i = 1$'den n'e, iç döngü $j = 1$'den i^2'ye gider. Her turda 1 eklendiği için

$$T = \sum_{i=1}^{n} \sum_{j=1}^{i^2} 1$$

yazılır. İçteki toplamda i^2 tane 1 vardır ve toplam j'ye bağlı değildir, yani sonuç i^2'dir. Hoca burada önce "i" deyip hemen düzeltti: Terim sayısı i değil i^2'dir. Geriye kareler toplamı kalır:

$$T = \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

3.2 `T = T + i`

Birin yerine `i` yazılırsa:

$$T = \sum_{i=1}^{n} \sum_{j=1}^{i^2} i$$

Hoca önce içteki toplamı "1'den i^2'ye kadar sayıların toplamı" gibi yazmaya başladı ve $\frac{i^2(i^2+1)}{2}$ ifadesine gitti. Bir öğrenci sonucun i^3 olması gerektiğini söyledi. Hoca bunu doğruladı ve teşekkür ederek hesabı düzeltti: Döngü değişkeni j'dir, içeride yazan i ise iç toplam için sabittir. O zaman i^2 tane i toplanır:

$$\sum_{j=1}^{i^2} i = i^2 \cdot i = i^3$$

Dıştaki toplam da küpler toplamı olur. Hoca bunun "biraz daha sevimli" bir formül olduğunu söyledi:

$$T = \sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

3.3 `T = T + j`

Şimdi içeride döngü değişkeni j vardır. İç toplam, 1'den i^2'ye kadar sayıların toplamıdır. Son terimi i^2 olan bu toplam için $\frac{\text{son terim} \cdot (\text{son terim}+1)}{2}$ kalıbı kullanılır:

$$\sum_{j=1}^{i^2} j = \frac{i^2(i^2+1)}{2}$$

$\frac{1}{2}$ dışarı alınıp ifade açılır:

$$T = \sum_{i=1}^{n} \frac{i^2(i^2+1)}{2} = \frac{1}{2} \left(\sum_{i=1}^{n} i^4 + \sum_{i=1}^{n} i^2 \right)$$

Kareler toplamını biliyoruz. Dördüncü kuvvetler toplamının formülü ise şudur. Hoca bunu "çok kötü bir formül" diye niteledi:

$$\sum_{i=1}^{n} i^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$$

Sonuç olarak:

$$T = \frac{1}{2} \left(\frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30} + \frac{n(n+1)(2n+1)}{6} \right)$$

> **Hocanın vurgusu:** Böyle bir soruda i^4'ün açılımı ya formül kağıdında verilir ya da soruda belirtilir.

Hoca, i^2, i^3, i^4 formüllerinin jenerik bir halinin, yani i^x için tek bir hazır formülün olmadığını söyledi. Bu formüllerin doğruluğunu kontrol etmek için bir yöntem göstermeyi de zaman kalırsa dersin sonuna bıraktı. Kayıtta bu yöntem gösterilmedi.

Ders sonrası not: Hoca i^x için jenerik bir hal olmadığını söyledi. $\sum i^k$ için büyük olasılıkla genel bir formül vardır (Faulhaber formülü, Bernoulli sayılarıyla yazılır), ama açık hali her k için farklı ve uzundur. Ders kapsamında $k = 1, 2, 3, 4$ formüllerinin kullanılması yeterlidir. Bu bölümdeki üç sonuç döngüler çalıştırılarak python ile doğrulandı.

4. `while` ile Katlanarak Artan Döngü

Yeni kod şudur:

```
T = 0
i = 1
while (i <= N)
    T = T + 1
    i = i * 2
end
```

4.1 Değişkenlerin izlenmesi ve terim sayısı

Hoca önce değişkenlerin nasıl ilerlediğini tabloyla izledi:

`i`	`T` (`T = T + 1` için)
1	1
2	2
4	3
8	4

`T`'yi hesaplarken ilk bakılacak şey toplam sembolünün sınırlarıdır. Döngüyü `i` kontrol eder. `i`, 1'den başlayıp n'e doğru gider, ama birer birer değil: 1, 2, 4, 8, yani $2^0, 2^1, 2^2, \ldots, 2^k$. Toplam sembolünün değişkeni ise otomatik olarak birer birer artar. Bu yüzden önce k'yi bulmak gerekir:

$$2^k = n \quad \Rightarrow \quad k = \log_2 n$$

Üsler 0'dan başlayıp k'ye kadar gittiği için terim sayısı k değil $k + 1$'dir, yani $\log_2 n + 1$.

4.2 `T = T + 1`

İçeriye ne yazılacağı da önemlidir. Burada `T` birer birer arttığı için içeriye 1 yazılır:

$$T = \sum_{i=1}^{\log_2 n + 1} 1 = \log_2 n + 1$$

Hocanın sayısal örneği: $n = 1024$ için $\log_2 n = 10$'dur, dolayısıyla `T` 11 bulunur (python ile doğrulandı).

4.3 `T = T + i`

`T + 1` yerine `T + i` yazılırsa desene yine değerlerle bakılır:

`i`	`T` (`T = T + i` için)
1	1
2	3
4	7

T'ye önce 1, sonra 2, sonra 4, sonra 8 eklenir. Yani eklenenler sırasıyla $2^0, 2^1, 2^2, \ldots$ değerleridir ve içeriye 2^i yazmak gerekir. Toplamı birden başlatsaydık ilk terim 2^0 olmazdı. Bu yüzden i'yi sıfırdan başlatıyoruz ve bunu dengelemek için üst sınırı 1 azaltıyoruz:

$$T = \sum_{i=0}^{\log_2 n} 2^i$$

Bu, i^2 ve i^3 gibi polinom bir fonksiyon değildir, üstel (eksponansiyel) bir fonksiyondur. Bu yüzden genel formülüne ayrıca bakmak gerekir.

5. Geometrik Seri Formülünün İspatı

Hoca en genel halini yazdı ve ispatladı. Tanım:

$$T = \sum_{i=0}^{n} a^i = a^0 + a^1 + a^2 + \cdots + a^{n-1} + a^n$$

Adımlar şunlardır:

- $a^0 = 1$ olduğu için 1 karşıya atılır:
  - $T - 1 = a^1 + a^2 + \cdots + a^n$
- Sağ taraf a parantezine alınır:
  - $T - 1 = a\left(1 + a + a^2 + \cdots + a^{n-1}\right)$
- Parantezin içi, T'nin son terimi olmayan halidir. Yani $1 + a + \cdots + a^{n-1} = T - a^n$:
  - $T - 1 = a\left(T - a^n\right)$
- Parantez açılır:
  - $T - 1 = aT - a^{n+1}$
- T'li terimler bir tarafta toplanır:
  - $aT - T = a^{n+1} - 1$
- T parantezine alınır:
  - $T(a - 1) = a^{n+1} - 1$

Böylece formül kağıtlarında görülen hale gelinir:

$$T = \frac{a^{n+1} - 1}{a - 1}$$

> **Ders sonrası not:** Hoca başta "a sıfırdan farklı" dedi. Son adımda $a-1$'e bölündüğü için asıl gereken koşul $a \neq 1$'dir (formül python ile doğrulandı). $a = 1$ iken toplam $n + 1$ olur.

5.1 Formülü bizim toplama uygulamak

Bizim toplamımızda $a = 2$, üst sınır da n yerine $\log_2 n$'dir:

$$T = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = 2^{\log_2 n + 1} - 1$$

Üstel ifade açılır: $2^{\log_2 n + 1} = 2^1 \cdot 2^{\log_2 n}$. Bir sayının, tabanı kendisi olan logaritma üssüne yükseltilmesi logaritmanın içindeki değeri verir, yani $2^{\log_2 n} = n$. Hoca bunun ispatına girmedi. Sonuç:

$$T = 2 \cdot n - 1 = 2n - 1$$

Ders sonrası not: $2n-1$ sonucu n 2'nin tam kuvveti olduğunda doğrudur. Döngü, n'i aşmayan son 2 kuvvetinde durur. Genel sonuç $2^{\lfloor \log_2 n \rfloor + 1} - 1$, tur sayısı da $\lfloor \log_2 n \rfloor + 1$'dir (ikisi de python ile doğrulandı). Dersteki $\log_2 n + 1$ ifadesi de aynı nedenle n'in 2'nin kuvveti olduğu durumu anlatır.

6. Polinom Toplam, Üstel Toplam ve $\sum i \cdot a^i$

Hoca bu noktada iki toplamı yan yana yazdı:

$$\sum_{i=1}^{n} i^2 \qquad \text{ve} \qquad \sum_{i=1}^{n} 2^i$$

Biri polinom, diğeri üstel fonksiyondur ve sonuçları birbirinden farklıdır.

Hocanın vurgusu: $\sum i^2$ ile $\sum 2^i$'yi karıştırmayın. Bu hata nedense çok yapılıyor.

Sonra başka bir durumu ele aldı: Bir algoritma yazdınız ve karmaşıklık hesabının sonunda şöyle bir toplam çıktı:

$$T = \sum_{i=0}^{n} i \cdot a^i$$

Burada doğrusal bir çarpan (i) ile üstel bir çarpan (a^i) birbiriyle çarpılıyor. Aradaki işlem toplama olsaydı iş kolay olurdu. Hoca önce sonucu yazdı ve "çok sıkıntılı, ama ispatı nispeten kolay" dedi:

$$T = \frac{1}{1-a}\left(\frac{a^{n+1}-1}{a-1} - 1 - n\,a^{n+1}\right)$$

İspat şu adımlarla yapılır. Önce toplam açılır:

$$T = 0 \cdot a^0 + 1 \cdot a^1 + 2a^2 + \cdots + (n-1)a^{n-1} + n\,a^n$$

İki taraf a ile çarpılır. Hoca bu adımı bir an hatırlayamadı, sonra her tarafın a ile çarpıldığını söyledi:

$$aT = 1 \cdot a^2 + 2a^3 + \cdots + (n-1)a^n + n\,a^{n+1}$$

Üstteki ifadeden alttaki çıkarılır. Aynı kuvvetli terimlerin katsayıları birer farklıdır: $2a^2 - a^2 = a^2$, $3a^3 - 2a^3 = a^3$ ve böyle devam eder. Geriye şu kalır:

$$T - aT = a + a^2 + a^3 + \cdots + a^n - n\,a^{n+1}$$

Sol taraf $T(1-a)$'dır. Sağdaki $a + a^2 + \cdots + a^n$ ise az önce bulduğumuz geometrik serinin $a^0 = 1$ terimi eksik halidir, yani $\frac{a^{n+1}-1}{a-1} - 1$'dir. Yerine yazılır ve $1-a$ bölüm olarak karşıya alınır (hoca bu son adımı hızlı geçti):

$$T(1-a) = \frac{a^{n+1}-1}{a-1} - 1 - n\,a^{n+1}$$

Böylece baştaki formül bulunur (python ile doğrulandı). Hoca bunların matematikçilerin ispat yöntemleri olduğunu, ama bir kodun analizinde böyle bir toplamla karşılaşılabileceğini söyledi. Sohbetten gelen yorum üzerine, kodu çalıştırıp sonucu görmenin bu formülle uğraşmaktan daha kolay görünebileceğini de not etti.

7. Bu Döngüler Kaç Kez Döner?

Hoca, karmaşıklıktan ne kastettiğini önceki iki örnekle açıkladı. `while` örneğinde `T + 1` da yazılsa `T + i` de yazılsa döngü $\log_2 n + 1$ kez döner. `T`'nin değeri yalnızca içeride ne yazdığına göre değişir. Başka bir ifadeyle, karmaşıklık için toplanan şeye değil döngünün dönme sayısına bakarız.

j sınırı i^2 olan iç içe döngüde de iki döngü birlikte $\frac{n(n+1)(2n+1)}{6}$ kez döner. Bu çarpım açılırsa n^3'lü bir terim gelir, yani karmaşıklık n^3'lü bir şeydir.

> **Ders sonrası not:** Hoca burada döngünün "en az n^3 defa" döneceğini söyledi. Doğrusu $\frac{n(n+1)(2n+1)}{6}$'dır ve baş terimi $\frac{n^3}{3}$'tür (python ile sayılarak doğrulandı). Yani mertebe n^3'tür, ama tur sayısı n^3'ten küçüktür.

8. `if` İçeren Döngü ve `else` Kolu

Yeni soruda `T`'nin artışı bir şarta bağlıdır:

```
T = 0
for i = 1:N
    for j = 1:i*i
        if mod(j, i) == 0
            T = T + 1
        end
    end
end
```

Burada `mod(j, i) == 0`, j'nin i'ye bölümünden kalanın sıfır olması demektir. Hoca şartı sağlayan j değerlerini tek tek yazdı:

i	j'nin gittiği yer	Şartı sağlayan j	T'ye eklenen	T
1	1	1	1	1
2	4	2, 4	2	3
3	9	3, 6, 9	3	6
4	16	4, 8, 12, 16	4	10 (ara adım eklendi)

> **Ders sonrası not:** Hoca $i = 1$ satırında "1'in 1'e bölümünden kalan 1'dir" dedi. Doğrusu kalan 0'dır (python ile doğrulandı). Şart bu yüzden sağlanır ve `T` 1 olur. Sonuç değişmez.

Her i için `T` birer birer artıyor gibi görünür, ama şart yüzünden o i turunda toplam i kadar artar. Hoca bu noktada önemli bir ayrıntıya dikkat çekti: İç toplamı

$$\sum_{i=1}^{n} \sum_{j=1}^{i^2} \; ?$$

olarak yazarsak soru işaretli yere `if`'li bir şey yazamayız. Bu şekilde terim toplamı çıkarılamaz. Yapılacak şey, tablodaki düzeni bulup toplamı ona göre yazmaktır:

$$T = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Soru: Girilen sayı kadar bölünen olmuyor mu? Mesela 5 girilirse 5 tane 5'e bölünen sayı var, T de 15 olur, mantık bu mu?

Cevap: Evet. Her i'de toplama i kadar ekliyoruz, n değeri için de yukarıdaki sonucu buluyoruz. (5 girildiğinde 15 sonucu python ile doğrulandı.)

8.1 `T = T + 1` else kolunda olsaydı

Slayt bir de şunu soruyor: `T = T + 1`, `if`'in içinde değil `else` kısmında olsaydı ne olurdu? Hoca bunu, olasılıkta çok anlatılan "tüm durumlar eksi istenen durum" mantığıyla çözdü. Bölünebilenler ve bölünemeyenler birlikte tüm durumları oluşturur:

$$T_{\text{else}} = T_{\text{tüm}} - T_{\text{if}}$$

Tüm durumlar, şart olmasaydı yapılacak sayımdır:

$$T_{\text{tüm}} = \sum_{i=1}^{n} \sum_{j=1}^{i^2} 1 = \sum_{i=1}^{n} i^2$$

Hoca burada da önce $\frac{i^2(i^2+1)}{2}$ yazıp "yok, öyle olmaz" diyerek düzeltti: i^2 tane terim vardır ve her biri 1'dir. Buradan $T_{\text{if}} = \sum i$ çıkarılır:

$$T_{\text{else}} = \sum_{i=1}^{n} (i^2 - i)$$

> **Ders sonrası not:** Hoca slayda bakarak "i kare eksi bir demiş" diye okudu. Tahtadaki çözüme ve python sayımına göre doğrusu $i^2 - i$'dir. Her i turunda bölünemeyen j sayısı $i^2 - i$'dir.

9. Üst Sınırı Önceden Hesaplanan Döngü

Hoca "işler biraz daha karmaşıklaşıyor" diyerek şu kodu açtı:

```
K = 1
for i = 1:N
    K = K * 2
end

T = 0
for i = 1:N
    for j = 1:K
        T = T + 1
    end
end
```

Toplam sembolleriyle yazalım:

$$T = \sum_{i=1}^{n} \sum_{j=1}^{K} 1$$

n'i kullanıcıdan alıyoruz, ama K'nin ne olduğunu bilmiyoruz. Üst limitte bir bilinmeyen olduğu için önce onu bulmak gerekir. Hoca `i` ile `K`'yi birlikte izledi:

`i`	`K`
1	2
2	4
3	8
n	2^n

Burada 0'dan başlayan bir üs yoktur. `i` 1'den başlar, `K` da 1'den başlayıp ilk turda 2 olur. Tek bir `for` döngüsü olmasına rağmen ulaşılan değer n tane ikinin çarpımı, yani 2^n'dir.

9.1 `T = T + 1`

$K = 2^n$ yerine yazılır. İç toplam 2^n tane birin toplamıdır. Dıştaki toplamda n tane 2^n vardır ve bu ifade i'ye bağlı değildir:

$$T = \sum_{i=1}^{n} 2^n = n \cdot 2^n$$

9.2 `T = T + i`

Sınırlar aynı kalır, yalnız içerisi değişir:

$$T = \sum_{i=1}^{n} \sum_{j=1}^{2^n} i = \sum_{i=1}^{n} i \cdot 2^n$$

2^n, i'ye göre sabit olduğu için dışarı alınır. Geriye 1'den n'e sayıların toplamı kalır:

$$T = 2^n \cdot \frac{n(n+1)}{2} = 2^{n-1} \cdot n(n+1)$$

> **Ders sonrası not:** Hoca sonucu "ya $2^n \cdot n \cdot (n+1)$ dersiniz ya da $2^{n-1} \cdot n \cdot (n+1)$" diye söyledi. Doğrusu yalnız $2^{n-1} \cdot n(n+1)$'dir. İlk ifade iki kat büyüktür, çünkü $\frac{1}{2}$ çarpanı düşmüştür (python ile doğrulandı).

9.3 `T = T + j`

Şimdi içteki toplam 1'den 2^n'ye kadar olan sayıların toplamıdır. Hoca bunu öğrencilerden sohbete yazmalarını istedi. Gelen cevap $\frac{2^n(2^n+1)}{2}$ oldu. Bu ifade i'ye bağlı değildir, doğrudan dışarı alınır ve içeride birler kalır:

$$T = \sum_{i=1}^{n} \frac{2^n(2^n+1)}{2} = n \cdot \frac{2^n(2^n+1)}{2}$$

Bu bölümdeki üç sonuç döngüler çalıştırılarak python ile doğrulandı.

10. Ara Sorular: Mantık Hatası, Global Değişkenler, Koşulda Atama

Aradan sonra hoca öğrencilerden gelen birkaç soruya baktı.

10.1 Sintaks hatası ve mantık hatası

Bir öğrencinin gönderdiği bubble sort kodu için hoca, öğrencinin "sintaks hatası" demesinin de ayrıca bir yanlış olduğunu söyledi. Kod çalışmasına engel bir şey içermiyorsa bu sintaks hatası değildir, muhtemelen mantıksal bir hatadır.

10.2 Global değişkenler neden kullanılmıyor?

Soru: Global değişkenleri neden kullanamıyoruz?

Cevap: Ayrıntısı üçüncü sınıftaki işletim sistemi dersinde görülür. Hoca kısaca iki gerekçe verdi.

Birinci gerekçe eş zamanlı (concurrent) kullanımdır. Windows gibi bir işletim sistemi arka planda çok sayıda programı çalıştırır. Bir programın kendisi de birden fazla kaynak tarafından eş zamanlı kullanılabilir. Global değişkenler bu durumda bir race condition'a girer.

Hocanın örneği banka hesabıdır. Bir uygulama hesaba para yatırmak (toplama), öbür uygulama para çekmek (çıkarma) istesin. Yazdığımız tek bir toplama sembolü makine tarafında `ADD`, `MOV` gibi 3-4 komuta dönüşür. İşletim sistemi işi toplama bitene kadar değil, o anki makine komutu bitene kadar sürdürür. Örneğin hesapta 5 lira var ve para ekleniyor. İkinci komut yapılırken işletim sistemi sırayı çıkarma işine verirse hesaptaki para yanlış görünür. Atomik olması gereken işlemlerde bu senkronizasyon sorunu yanlış sonuç üretir.

Bir kaynağa böyle erişim varsa ya kaynak paylaşılmaz (non-shareable) yapılır ya da paylaşılan değişkene (shared variable) erişim semafor, monitör gibi karşılıklı dışlama (mutual exclusion) teknikleriyle düzenlenir. Böylece bir program değişkeni aldığında işi bitene kadar diğeri ona erişemez. Hocaya göre bu dersin seviyesinde çözüm, global değişken kullanmamaktır. Çok thread'li ve çok fonksiyonlu karmaşık uygulamalarda global değişken kullanılmaz.

İkinci gerekçe tasarımdır. Parametre aktarımı, yani call by value ve call by reference, işin mühendislik tarafıdır. Global değişken kullanmak "tekniker seviyesinde" daha basit bir yoldur. Gerektiğinde kullanılabilir, ama istenen tasarım, değişkenlerin fonksiyonlara parametre olarak gitmesi ve gerekirse fonksiyondan dönmesidir.

> **Ders sonrası not:** Hoca, bir programın eriştiği değişkeni başkasının ele geçirememesi durumuna "preemptive deniyor herhalde" dedi. Bu durum büyük olasılıkla karşılıklı dışlamadır (kilitleme). Preemptive terimi büyük olasılıkla bunun tersini, işletim sisteminin çalışan işi yarıda kesip sırayı başkasına vermesini anlatır. Ayrıca bir global değişkeni büyük olasılıkla aynı programın thread'leri paylaşır. Ayrı programların bellekleri genelde birbirinden ayrıdır.
>
> **Hocanın vurgusu:** Bu derste global değişken, `switch` dışında `break`, `continue` ve `goto` kullanımı yasaktır. `static` ise izne bağlıdır.

10.3 `for` koşulunda atama

Bir öğrencinin kodunda `expression is not assignable` hatası çıkıyor, düzeltilmiş halinde de program ne girilirse girilsin hep 2 yazıyordu. Hoca `for` koşulunda `j = n-i-1` yazıldığını gördü. Burada istenen karşılaştırmadır ve `==` yazılmalıdır. `=` ile `j`'ye sürekli bir değer atanır. Bu değer 0'dan farklı olduğu sürece, 1 de olsa 5 de olsa, koşul doğru sayılır. Hoca, `x = -1` yazmanın `x = 1` yazmaktan bu açıdan farkı olmadığını da ekledi.

11. Üç İç İçe Döngü

Hoca normalde bu kodları C ile yazıp sonuç gösterdiğini, ama burada gerek olmadığını söyledi:

```
T = 0
for i = 1:N
    for j = 1:i
        for k = 1:j
            T = T + 1
        end
    end
end
```

Sınırlar içeriye doğru birbirine bağlıdır, ama neyse ki hepsi birer birer ilerler:

$$T = \sum_{i=1}^{n} \sum_{j=1}^{i} \sum_{k=1}^{j} 1$$

Önce en içteki toplam çözülür. j tane 1 vardır, yani sonuç j'dir. Bir sonraki seviye 1'den i'ye kadar sayıların toplamıdır:

$$\sum_{j=1}^{i} j = \frac{i(i+1)}{2} = \frac{i^2 + i}{2}$$

Hoca burada önce j yazmaya başlayıp düzeltti: Değişken ve terim sayısı artık i'dir. $\frac{1}{2}$ dışarı alınır:

$$T = \frac{1}{2}\left(\sum_{i=1}^{n} i^2 + \sum_{i=1}^{n} i\right) = \frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right)$$

Dıştaki 2 paydalara dağıtılınca hocanın vardığı sonuç gelir:

$$T = \frac{n(n+1)(2n+1)}{12} + \frac{n(n+1)}{4}$$

Hoca burada bıraktı. İfade tek kesirde toplanırsa $\frac{n(n+1)(2n+1)+3n(n+1)}{12} = \frac{n(n+1)(2n+4)}{12} = \frac{n(n+1)(n+2)}{6}$ olur (ara adım eklendi, python ile doğrulandı).

Hoca bu konuda sayfada çalışma sorusu bulunmadığını, soruları kendinizin üretmeniz gerektiğini söyledi. En içte 1 yerine `i`, `j` veya `i + j` yazılabilir. `i + j` için ilk adımı da gösterdi: k'ye göre $i + j$ sabittir, dolayısıyla en içteki toplam j tane $(i + j)$ eder:

$$\sum_{k=1}^{j}(i + j) = j(i + j) = ij + j^2$$

Sonra $\sum_{j=1}^{i} ij$ ile $\sum_{j=1}^{i} j^2$ ayrı ayrı düşünülür ve sonuç i'ye göre açılarak devam edilir. Hoca sonucu burada bıraktı ve kodunu yazıp denemeyi önerdi.

12. Alt Sınırı 3 Olan Döngü ve Karmaşıklık Hesabı

Son analiz sorusu görece kolaydır:

```
T = 0
for i = 1:N
    for j = 3:N
        T = T + 1
    end
end
```

Slaytta ayrıca iç döngü `3:N` yerine `i:N` olsaydı ne olacağı da soruluyor. Bu varyasyon derste çözülmedi.

12.1 `T`'nin değeri

Hoca bu soruyu öğrencilere sorarak çözdü. Sınırlar $i = 1$'den n'e ve $j = 3$'ten n'e yazılır.

Soru (hocadan): Toplamın içine ne yazacağız?

Cevap: Bir öğrenci j dedi. Hoca kodu izleyerek bunun neden olmadığını gösterdi. `T` 0'dan başlar, $n = 5$ alınsa bile döngüye ilk girişte `T` 1 olur. İçeriye j yazsaydık ilk değer 3 olurdu, sonra 4 eklenirdi. Oysa kodda her turda 1 ekleniyor. O yüzden içeriye 1 yazılır:

$$T = \sum_{i=1}^{n}\sum_{j=3}^{n} 1$$

İç toplamın terim sayısı "son terim eksi ilk terim artı 1" kuralıyla bulunur: $n - 3 + 1 = n - 2$. İçeride j'ye bağlı bir şey olmadığı için bu sayı doğrudan dışarı çıkar. Dış toplamda n tane $(n - 2)$ vardır:

$$T = \sum_{i=1}^{n} (n-2) = n(n-2)$$

Burada bir öğrenci önce $(n-2)(n-1)$ dedi, sonra terim sayısının n olduğunu görüp $n(n-2)$'ye vardı.

> **Ders sonrası not:** Hoca sonucu söylerken bir an "n kare eksi 2" dedi, sonra "n tane $n-2$" diye açıkladı. Doğrusu $n(n-2) = n^2 - 2n$'dir (python ile doğrulandı).

12.2 Temel işlem toplama iken karmaşıklık

Hoca bir öğrenciye bu kod parçasının karmaşıklığını sordu. Temel işlem (basic operation) olarak toplama alınırsa `T = T + 1` satırının kaç kez çalışacağına bakılır. Bu satırı çalıştıran iki döngü vardır ve sınırları yukarıdakilerin aynısıdır. Toplama sembolünün maliyetine 1 birim dersek:

$$C_{\text{toplama}}(n) = \sum_{i=1}^{n} \sum_{j=3}^{n} 1 = n(n-2)$$

Yani toplama işlemi $n(n-2)$ kez yapılır. Bir bilgisayarda bir toplamanın süresi biliniyorsa programın toplama için harcadığı süre kabaca bu sayıyla çarpılarak bulunur. Hocanın sayısal örneği şudur:

- **Verilen:** $t_{\text{toplama}} = 0{,}0001$ saniye, $n = 10^6$.
- **İşlem:** $10^6 \cdot (10^6 - 2) \cdot 0{,}0001$.
- **Ara sonuç:** $10^6 \cdot 0{,}0001 = 10^2$.
- **Sonuç:** $10^2 \cdot (10^6 - 2)$ saniye, yani yaklaşık 10^8 saniye.

> **Ders sonrası not:** Hoca çarpımı söylerken bir an "0,001" dedi, ama ara sonuç olarak verdiği 10^2 yalnız 0,0001 ile tutar. Süre yaklaşık 10^8 saniyedir, bu da yaklaşık 3,2 yıl eder (python ile hesaplandı).

12.3 `T + 1` ile `T + j` arasında maliyet farkı var mı?

Soru: Toplamın içinde 1 yerine j olsaydı karmaşıklık sonucu aynı olur muydu? Sonuç farklı çıkıyor.

Cevap: Sonucun farklı olması `T`'nin değerinin farklı olmasıdır. Karmaşıklık derken süre veya işlem sayısı gibi bir analiz yaparız. Toplama, bir birim zamanda yapılan bir işlemdir. `T + j` de bir kez toplamadır, `T + 1` de.

Soru: `T + 1` için bir kez toplarken, `T + j` için j eklemek amacıyla j kez toplamıyor muyuz?

Cevap: Hayır. İşlemci `T`'ye 1 eklerken de 5 eklerken de bir kez toplama yapar. İki değer de tam sayıdır, ikisi de 4 byte yer kaplar. Yani 32 bit karşılıklı olarak toplama devresine girer. 1 toplamakla 5 toplamak arasında fark yoktur. Biri tam sayı, biri kayan noktalı olsaydı iş değişirdi, çünkü kayan noktalı hesap devreye girerdi.

Soru: Yani `T + 1` de yazsak `T + j` de yazsak karmaşıklık hesabında içeriye 1 mi yazacağız?

Cevap: Evet. Temel işlem toplama ise toplamanın sağındaki ve solundaki işlenenlere (operand) dikkat etmen gerekmez. Oradaki 1, `T + 1`'deki 1 değildir, bir tane toplama işlemini gösterir. Satırda iki toplama olsaydı 2 yazılabilirdi. Yaptığımız iş, toplama işlemlerinin adedini bulmaktır. Bu bir sayma (count) problemidir.

12.4 Temel işlem atama iken

Hoca aynı kodda atamaları da saydı:

- `T = 0`: 1 atama.

- `i = 1`: 1 atama.
- `j = 3`: n atama. İç döngüye her girişte yapılır.
- `T = T + 1`: $n(n-2)$ atama.

Toplarsak bir fonksiyon elde edilir. Bu sayı atamanın birim süresiyle çarpılınca atamaların süresi hesaplanır. Bütün programı analiz etmek istersek atama, karşılaştırma, toplama, çıkarma ne varsa hepsini böyle tek tek yazıp uzun bir fonksiyon elde etmemiz gerekir.

> **Ders sonrası not:** Hoca toplamı "$n^2 - n + 3$ gibi bir şey" olarak söyledi. Yukarıdaki dört satırın toplamı $n^2 - n + 2$'dir (python ile sayılarak doğrulandı). Döngü değişkenlerinin artırılması da atama sayılırsa sonuç büyük olasılıkla daha büyük olur. Mertebe her durumda n^2'dir.

13. Rastgele Sayılara Giriş: Çember Üzerinde Desen

Hoca, analiz konusu bittiği için yarınki konuya, rastgele sayılara (random) bir giriş yaptı. Girişi şekil üreten bir algoritmayla yaptı. Slaytta bir çember üzerinde eşit aralıklı noktalar ve bu noktaları birleştiren çizgilerden oluşan desenler vardı. Bir noktadan başlanıyor, başka bir noktaya çizgi çekiliyor, oradan bir başkasına gidiliyordu. Farklı başlangıçlarla ve farklı değerlerle bazıları çok güzel, bazıları çirkin desenler oluşuyordu.

Hoca, çizimin C kodunun şu an yazılamayacağını, fikri kaba kod veya akış şeması üzerinden anlattığını belirtti. Çizgi çizmenin nasıl yapıldığını şimdilik sormamamızı, hazır kod olduğunu varsaymamızı söyledi. Şekil çizdirmek için grafik kütüphaneleri kullanılır.

Algoritmanın akışı şudur:

1. Başla. Çember üzerindeki nokta sayısı m ve rastgelelik sağlayan sabit s kullanıcıdan alınır.
2. `NoktaÜret(m)` çağrılır. Bu fonksiyon, m noktanın x ve y koordinatlarını tutan `P` matrisini döndürür.
3. Başlangıç noktası seçilir. Hoca, rastgele başlatmak istemediği için `i = 1` aldı.
4. `j` 1'den m'e birer birer gidecek şekilde m kez şunlar yapılır:
   - Yeni nokta hesaplanır: `yi = (i * s) mod m`.
   - `Birlestir(P, i, yi)` ile `i` ve `yi` noktaları arasına çizgi çekilir.
   - Bir sonraki adım için `i = yi` yapılır.
5. Bitir.

Değişkenlerin anlamı şöyledir: `i` şu anki nokta, `yi` bir sonraki noktadır. m'ye göre mod almak, sonucun 0 ile $m-1$ arasında çıkması, yani m noktadan birine gidilmesi demektir. s, keyfi seçilmiş (arbitrary) bir sabittir.

`NoktaÜret` fonksiyonunun yaptığı iş de şudur: Çemberin bir merkez koordinatı ve bir yarıçapı belirlenir. Merkezden yarıçap kadar uzaktaki noktalardan m tane eşit aralıklı nokta üretilir. Her noktanın genel koordinat sistemine göre bir x ve bir y değeri vardır ve bu değerler `P` matrisinde saklanır.

> **Ders sonrası not:** Hoca kuralı ilk yazarken "`yi = yi * s mod m`" dedi, sonra iki kez `yi = i * s mod m` olarak okudu. Doğru olan ikincisidir, çünkü yeni nokta o anki `i` noktasından hesaplanır. Ayrıca mod işlemi 0 ile $m-1$ arasında değer üretir, hoca ise noktaları 1'den m'e de numaralandırdı. Kodlanırken numaralama büyük olasılıkla 0'dan başlatılmalıdır.

14. Tekrar Sorunu

Hoca sonra algoritmanın bir sıkıntısını gösterdi. Diyelim ki noktalar arasında şöyle gidildi: 1, 3, 5, 7, 8 ve sonra yine 1. 1 noktasına tekrar gelindiğinde bir sonraki nokta ne olur? Yine 3'tür. Bunun kaçarı yoktur, çünkü yeni noktayı `i`, `s` ve m belirler. s ve m sabittir, `i` de bir önceki değerdir. Döngü değişkeni `j` işleme girmediği için farklı bir noktaya gitme olasılığı yoktur.

Başka bir durum da şudur: 7'den 1'e değil 3'e gelinseydi, dizi 1, 3, 5, 7, 3, 5, 7, 3, … diye giderdi. Program bu noktalar arasında kendi içinde dönüp dururdu.

Soru: İkinci turda tekrar 1'e uğraması gerekmiyor mu?

Cevap: Hayır. 1'den başladık, 3, 5, 7'ye gittik, 7'den de 3'e geldik. 3'ten sonrası hep aynı olduğu için 1'e bir daha dönülmez.

Demek ki işlemi öyle yapmak gerekir ki sıra kısa döngüler içermesin. Döngü olacaksa bile çok uzun bir işlemden sonra, olabilen bütün noktalar gezildikten sonra başa dönülsün. Dairenin bütün noktalarına uğramak için `i`'nin çok sayıda tekil değer alması gerekir. Bir noktaya gelip içeride bir döngüye girilirse desen tekrara düşer.

> **Ders sonrası not:** 1, 3, 5, 7, 8 sırası büyük olasılıkla tekrar fikrini göstermek için söylenmiş temsilî bir sıradır, `(i * s) mod m` kuralının ürettiği gerçek bir dizi değildir. 1'den başlayan bu çarpımsal kural yalnız s'nin kuvvetlerini gezer. $\gcd(s, m) = 1$ iken 0 noktasına ve m ile ortak böleni olan noktalara hiç gelinmez (python ile denendi). $\gcd(s, m) > 1$ iken de dizi kısa bir döngüye girebilir ya da 0'a düşüp orada kalabilir (bu da python ile denendi).

15. Uğrama Sayıları ve Tek Düze Dağılım

Hoca, bir noktaya kaç kez uğrandığını nasıl bilebileceğimizi sordu. Bir öğrencinin önerisi doğruydu: Dizide saklayabiliriz. Elemanları başta sıfır olan, her nokta için bir gözü bulunan bir `T` dizisi tutulur. Bu dizi aslında bir frekans dizisi, yani histogramdır. Döngüde her noktaya gelindiğinde `T[i]` 1 artırılır. Böylece hangi noktaya kaç kez uğrandığı saklanır. Önceki derslerde yapılan eleman sayma işleminin aynısıdır.

Her noktaya eşit sayıda uğranıyorsa, örneğin dizi 4, 4, 4, 4, ... ise burada tek düze (uniform) bir dağılım vardır. Kaç değer üretilirse üretilsin dairenin çevresi eşit biçimde dolaşılır. Desenler karmaşıklaşır, ama bütün noktalara aynı sayıda ulaşılmıştır. Hoca bir noktaya daha fazla, diğerlerine daha az uğratmanın yoluna daha sonra geleceklerini söyledi.

Tek düze dağılım, her sayının ulaşılma olasılığının eşit olması demektir. m nokta varsa her birinin olasılığı $\frac{1}{m}$'dir.

Soru (hocadan): Bu durumda üretilen bir sayının tahmin edilebilirliği minimum mu, maksimum mu olur?

Cevap: Minimumdur. Bütün olasılıklar eşit olduğu için "kesinlikle şu gelecek" denebilecek bir değer yoktur. 3 de gelebilir, 5 de. Söylenen her değerin olasılığı aynı $\frac{1}{m}$'dir.

Hoca karşılaştırma için bir histogram çizdi: 1'den 6'ya kadar değerler, ama 6'nın gelme sıklığı diğerlerinden yüksek. Bu dağılımda 6'yı tahmin etmek daha kolaydır, çünkü gelme olasılığı daha yüksektir. Tek düze dağılımda ise "ne söylesen boş"tur.

16. Gerçek ve Sözde Rastgele Sayı Üreteçleri

Hoca, `yi = (i * s) mod m` kuralındaki s sabit olmasaydı ve her adımda rastgele üretilseydi noktaların kendini tekrar etme olasılığının azalacağını söyledi. Peki bilgisayarda bu rastgelelik nasıl sağlanır? Rastgelelik birkaç sınıfta ele alınır ve rastgele sayı üreteçleriyle (random number generator) yapılır. Kriptoloji ve ayrık olay simülasyonu dersleri bu konuyla ilgilenir.

Gerçek rastgele sayı üreteci (true random number generator): Hocaya göre sayısal bilgisayarlarda gerçekte yoktur. Burada bir noktadan başlanır, ilerlenir ve dizi sürekli rastgele devam eder. Bir değere tekrar gelinse bile üretilen değerler birbirine bağıntılı olmadığından sonraki değer tekrar etmeyebilir. Örnek: Hilesiz bir parayı, hava gibi etkileyici hiçbir şartın olmadığı bir durumda bin kez atıp 0-1 sonuçlarını dizi olarak kaydetmek. Zar için de aynısı söylenebilir.

> **Ek bilgi (derste anlatılmadı):** Günümüzdeki işlemcilerin bir kısmında büyük olasılıkla ısıl gürültü gibi fiziksel olaylardan yararlanan donanımsal rastgele sayı kaynakları bulunur. Dersteki "bilgisayarda yok" ifadesi, yalnız yazılımla çalışan ve bir formüle dayanan üreteçler için geçerli olarak okunmalıdır.

Sözde rastgele sayı üreteci (pseudo random number generator): Tekrar sıklığı çok düşük diziler üretmeye çalışır. Basit bir örnek şu fonksiyondur. n. terim, önceki terimden üretilir:

$$r_n = (a \cdot r_{n-1} + c) \bmod m$$

Başlangıç değeri r_0'ı vermek zorundayız. Hocanın örneğindeki değerler şöyledir: $r_0 = 5$, $a = 1366$, $c = 150889$, $m = 714025$.

Bu dizi illa ki bir yerde kendini tekrar eder. r_0'dan tekrara kadar olan uzaklık ne kadar fazlaysa üreteç o kadar iyidir.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki değerlerle dizi python ile üretildi ve $r_0 = 5$'e ancak 714025 adımda dönüldü. Yani dizi m'nin izin verdiği en uzun tekrar uzunluğuna ulaşır. Bunun büyük olasılıkla sebebi, a, c ve m'nin Hull-Dobell koşulları denen şartları sağlamasıdır.

17. ANSI C Üreteci ve Çekirdek Değer

Hoca, C'de daha önce yaptıkları random örneğinde "bunu şimdi sormayın, sonra anlatacağım" dediği konuya döndü. ANSI C'nin üreteci için slayttaki ifade şudur:

$$s_0 = 12345, \qquad s_{i+1} = (1103515245 \cdot s_i + 12345) \bmod 2^{31}$$

s_0, önceki örnekteki r_0'ın karşılığıdır ve çekirdek, yani temel değer (seed) olarak adlandırılır. s_{i+1} önceki değerlerden türer. Bu türeme s_i'den, s_{i-1}'den ya da s_{i-2}'den olabilir, ona uygun bir fonksiyon yazmak gerekir.

Siz hiçbir şey yapmazsanız, program bu çekirdekle bir sayı üretir. Hocanın sözlü örneği: Program çalıştı ve 5 üretti. Bu 5 formülde yerine yazıldı, mesela 125 üretildi, sonra bu değerden bir sonrakine geçildi. Programı kapatıp tekrar çalıştırdığınızda yine 5'ten başlar, çünkü kodda her şey sabittir ve farklılık yaratacak bir etki yoktur. Her çalıştırmada aynı dizi görülür.

> **Ders sonrası not:** Hoca, slayta göre çekirdeğin 12345 olduğunu söyledi. Formüldeki 12345 artım terimidir. C standardında `srand` çağrılmazsa üreteç büyük olasılıkla `srand(1)` çağrılmış gibi başlar. Burada gcc ve glibc ile denendi: `srand` çağırmadan alınan ilk `rand` değeri `srand(1)` sonrası alınanla aynı, `srand(12345)` sonrası alınandan farklı çıktı. Gerçek kütüphane kodunun formülü büyük olasılıkla derleyiciye göre değişir.

18. `srand(time(NULL))`

Her çalıştırmada aynı diziyi görmemek için daha önce `main`'in hemen altına `srand(time(NULL))` yazılmıştı. `srand`, çekirdek değeri değiştiren fonksiyondur. İçine 3 verilirse çekirdek 3, 10 verilirse 10 olur. Ama içine sabit bir sayı yazılırsa, hatta `x` gibi her çalıştırmada aynı değeri alan bir değişken verilirse, program her çalıştığında yine aynı dizi görülür.

O halde içine sürekli değişen bir şey vermek gerekir. Bunun için zaman fonksiyonu kullanılır. `time`, 1 Ocak 1970'ten programın çalıştırıldığı ana kadar geçen saniyeyi verir. Hocanın örneği: Bu değer şu an 3 milyarsa, 3 saniye sonra 3 milyar artı 3'tür. Zaman geri döndürülemeyeceği için çekirdek her çalıştırmada farklıdır ve her seferinde farklı diziler görülür. Bu, sözde rastgele sayı üretecinin kullanımına bir örnektir.

Daha önce yazılan kullanım şöyledir (`% 10` hocanın hatırlattığı kısımdır, kodun kalanı bu anlatımdan kuruldu ve `gcc -Wall` ile uyarısız derlendi):

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    int i;

    srand((unsigned) time(NULL));   /* çekirdek bir kez atanır */
```

```
    for (i = 0; i < 5; i++) {
        printf("%d\n", rand() % 10);
    }
    return 0;
}
```

`srand`'ın bir defa yazılması yeterlidir. Bununla sayı üretmiyoruz, yalnız çekirdek değerini değiştiriyoruz. Tekrar yazılırsa çekirdek yeniden değişir ve dizi yeni bir s_0'dan başlar. `rand` ise istenildiği kadar çağrılır. İlk çağrıdaki değer çekirdeğe göre, sonraki çağrıdaki değer önceki değere göre elde edilir ve bu değer bir yerde saklanır.

Soru (sohbetten): `srand` makro mu? Tekrar çağrılırsa çekirdek her saniye değişmez mi?

Cevap: `srand` makro değil, fonksiyondur. İçine bir değişken yazılır. Belki `time` makro olabilir, ama fark etmez. Her saniye değişen bir şey de yoktur. `time` çağrıldığı anın değerini alır ve onu çekirdek yapar.

> **Ders sonrası not:** Hoca `time`'ın belki makro olabileceğini söyledi. `time` büyük olasılıkla `<time.h>` içinde tanımlı standart bir fonksiyondur. Hoca `NULL`'dan "o an, now" diye de söz etti. `NULL` büyük olasılıkla o anı göstermez, sonucun ayrıca bir değişkene yazılmayacağını belirtir. Zaman değerini fonksiyonun kendisi döndürür.

Bu bölümün sonunda hoca şunu özetledi: m değeri ne kadar büyükse "rastgelelik illüzyonu" o kadar iyi olur, yani gördüğümüz sayılar gerçek rastgeleliğe o kadar yakınlaşır. Büyük m neyi görmemizi engeller? Döngüyü. Bir dizide 1, 2, 6, 8, 7 gibi gidip tekrar 1 görürsek dizinin kendini tekrar ettiğini anlarız. m ve diğer parametreler öyle iyi seçilmelidir ki bu tekrar çok uzun olsun ve deseni göremeyelim.

Sonraki derste rastgele sayı üretimi kod üzerinden devam edecek. Konular zar örneği, 52 kart, Gauss dağılımı, robot örneği ve pi sayısının bulunmasıdır.

19. Ders Sonu Soruları: `srand` ve Kütüphane Kodları

Soru: `srand`'ı `for`'un içine koyunca hep aynı değerleri vermeye başlıyordu. Neden?

Cevap: Hoca "Hayır olmaz, bir hatanız var demektir" dedi ve konuyu sonraki derste kodlayacaklarını söyledi. `srand` `main`'in hemen altında bir defa çağrılır, ardından her `rand` çağrısında farklı sayılar gelir. Hocaya göre çok yapılan hata `srand`'ı `for` içine yazmaktır. Bu durumda değerlerin yine farklı görüneceğini, ama bunun gereksiz bir işlem olduğunu ekledi.

> **Hocanın vurgusu:** `srand` bir defa set edilir, `rand` istediğiniz kadar çağrılır.

> **Ders sonrası not:** Öğrencinin gözlemi doğrudur. `srand(time(NULL))` döngüye konursa ve döngü aynı saniye içinde dönerse çekirdek her turda aynı değere döner, ilk `rand` değeri de hep aynı çıkar. Burada gcc ile denendi: Döngüdeki bütün `rand() % 10` değerleri aynı çıktı. Yani bu kullanım gereksiz olmanın ötesinde hatalıdır.

Soru: `srand` ve `rand`'ın içinde ne olduğunu nasıl görebilirim?

Cevap: Bunlar standart kütüphanededir ve "inside the srand function" gibi aramalarla araştırılabilir. Kütüphanenin içindeki yazılmış kodlara bakılabilir, bilgisayarınızda da vardır.

> **Ders sonrası not:** Hoca bu fonksiyonları "`stdio.h` veya standart kütüphane" içinde saydı. `rand` ve `srand` büyük olasılıkla `<stdlib.h>`, `time` ise `<time.h>` içindedir. Başlık dosyalarında büyük olasılıkla yalnız bildirimler bulunur. Kaynak kodu için açık kaynaklı C kütüphanelerine, örneğin GNU C Library'ye bakılabilir.

20. Seyrek Matris Kodunda Hata Arama ve Dev-C++ Debug Kullanımı

Dersin sonunda bir öğrenci laboratuvar sorusuna yazdığı kodu gösterdi. Bu soru 23. bölümde anlatılmıştır. Programın ikinci menüsünde sıkıştırılmış matristen asıl matris oluşturuluyordu. Satır sayısı olarak 3 ve altı girildiğinde çalışıyor, 4 ve üzeri girilince sonuç vermiyordu.

20.1 Hocanın kodda gördükleri

- Matris `MAX` değerine göre yazdırıldığı için ekranda çok sayıda sıfır görünüyordu. Matris, gerçek satır ve sütun sayısına göre yazdırılmalıdır.
- Sıkıştırılmış matrisin üç sütunu vardır: iki sütun koordinatı, biri değeri tutar. Bu yüzden iç döngünün üç kez dönmesi normaldir.
- Değer yerleştirme atamasının üç sütunluk `for`'un içinde değil, dışında olması daha doğrudur. Hoca hatanın bununla ilgili olmadığını, ama doğruluk açısından buna dikkat edilmesi gerektiğini söyledi.
- Kodda boş bir `else` vardı. Hoca "Kullanacaksan yazacaksın, kullanırım diye yazılmaz" diyerek silmesini istedi.

Öğrenci, aynı kodun başka bir bilgisayarda Linux üzerinde segmentasyon hatası verdiğini de söyledi. Hoca bunun üzerine muhtemelen matrisin dışına taşıldığını söyledi. `for` döngülerinin sınırları ve değişkenleri kontrol edilmelidir. Hata derste kesin olarak bulunamadı, öğrenciye kontrol edip gerekirse kodu göndermesi söylendi.

20.2 Debug nasıl yapılır?

Bu inceleme ve hemen ardından başka bir öğrencinin ekranı üzerinden Dev-C++'ta debug kullanımı görüldü. Adımlar şunlardır:

1. **Projeyi açmak:** Bu sürümde debug için dosyanın proje olarak açılması gerekebilir. Tek `.c` dosyasıyla çalıştırınca program debug'a girmeden kapanıyordu. Çözüm olarak yeni bir proje açılıp kod onun `main.c` dosyasına kopyalandı. Hoca, açık Dev-C++ pencerelerinin hepsinin kapatılması gerektiğini ve projenin dosyasından açılmasını söyledi. Kendi kullandığı 5.9.1 sürümünde ise buna gerek olmadığını belirtti.
2. **Breakpoint koymak:** İncelenecek satıra breakpoint konur.
3. **Derlemek:** Kod değiştiyse önce derlenir, çünkü debug son derlenmiş haliyle çalışır.
4. **Debug'ı başlatmak:** Execute menüsünden veya aşağıdaki panelden Debug seçilir. Program çalışıyorsa önce Stop Execution gerekebilir.
5. **Adım adım ilerlemek:** `Next line` sonraki satıra geçer, `Into function` çağrılan fonksiyonun içine girer, `Continue` sonraki breakpoint'e kadar çalıştırır. Uzun yazdırma döngülerini hızla geçmek için `Continue` kullanışlıdır.
6. **Girdi vermek:** Sıra `scanf`'e gelince program girdi bekler. Komut satırı penceresine değer girilir, sonra `Next line` ile devam edilir.
7. **Değişkenleri izlemek:** Fare işaretçisi değişkenin üzerine getirilip beklenir ya da değişken seçilip `Add watch` denir.

20.3 Insertion sort sorusu

Bir öğrencinin insertion sort kodu derleniyor, ama yanlış sıralıyordu. Sorun mantıktaydı, sintaksta değil. Hoca, döngünün sıfırdan başladığını gördü ve "Insertion sort sıfırdan başlamaz, birden, yani ikinci elemandan başlar" dedi. Geri kalan sınırları kontrol etmesini söyledi.

21. Polinom Koduna Dönüş

Hoca, ders başında bakılan polinom koduna döndü. Öğrenci hatanın kaynağını nasıl araştırabileceğini sordu. Polinom $x^2 + x + 1$ idi. Hesabın yapıldığı satıra breakpoint kondu ve hoca, program oraya gelmeden devam ettiği için ekranı devralıp denedi.

Girişte `x` long float (`%lf`) ile alınıyordu. Hoca, değişkenleri ve fonksiyonun dönüş tipini `float`, biçimleri `%f` yaptı, prototipi de değiştirdi. Bu denemede $x = 5$ girildiğinde de aynı çıktı görüldü ve sebep derste bulunamadı. Hoca, algoritmada bir sorun olmadığını, burada büyük bir sayının da bulunmadığını söyledi. Bir öğrenci benzer çıktıyı çok büyük ve çok küçük değerlerde gördüğünü söyledi. Hoca kodu ders sonrası incelemek üzere aldı. Sürümden veya bilgisayardan kaynaklanabileceğini de ihtimal olarak ekledi.

> **Ek bilgi (derste anlatılmadı):** Eski Windows çalışma kütüphanesi (msvcrt) büyük olasılıkla sonsuz değerini `1.#INF` olarak yazar. `%.3f` ile yuvarlanınca bu yazım büyük olasılıkla `1.#IO`'ya dönüşür. Altyazıdaki "bir çarpı IO" da büyük olasılıkla budur. Yani hesapta büyük olasılıkla sonsuz değeri oluşuyordu. Sebep olarak büyük olasılıkla tip

uyuşmazlığına (`scanf`'te `double` için `%lf`, `float` için `%f` gerekir) veya bir ara değerin ilk değer almamasına bakılmalıdır.

22. `exchange` Değişkeni ve `do-while`

Laboratuvar bölümünün başlamasını beklerken hoca, başka bir öğrencinin bubble sort kodundaki iki sorunu söyledi. `exchange` değişkeni 0 ile başlatılıyor, `while` tarafında ise 1'e atanıyordu. Hoca, koşulda 1'e atama yapılırsa zaten sonsuz döngüye girileceğini söyledi. Çözüm iki türlüdür: `exchange` 1 ile başlatılır ya da `do-while` kullanılır.

Hocanın söylediği iki çözüm, sıralama döngüsüyle birlikte şöyle yazılır (kalıp bu anlatımdan kuruldu, `gcc -Wall` ile uyarısız derlendi ve çalıştırılarak sınandı):

```c
void kabarcik_while(int a[], int n)
{
    int i, gecici;
    int exchange = 1;

    while (exchange == 1) {
        exchange = 0;
        for (i = 0; i < n - 1; i++) {
            if (a[i] > a[i + 1]) {
                gecici = a[i];
                a[i] = a[i + 1];
                a[i + 1] = gecici;
                exchange = 1;
            }
        }
    }
}

void kabarcik_do_while(int a[], int n)
{
    int i, gecici;
    int exchange;

    do {
        exchange = 0;
        for (i = 0; i < n - 1; i++) {
            if (a[i] > a[i + 1]) {
                gecici = a[i];
                a[i] = a[i + 1];
                a[i + 1] = gecici;
                exchange = 1;
            }
        }
    } while (exchange == 1);
}
```

Öğrenci `do-while` yazmayı hiç kullanmadığını söyleyince hoca bu yapıya alışmak gerektiğini anlattı. Örneği ikili arama (binary search) idi. İnternetteki kodlar hep `while` ile yazılmış, ama aranan değerin ilk denemede bulunma olasılığı vardır ve bu iş `do-while`'a daha iyi oturur. `while` ile orta nokta bulma veya bir atama gibi işleri koda iki kez yazmak gerekir. `do-while`'da gerekmez. Giriş (login) işlemleri de bu döngü için iyi bir örnektir. Hoca bazı dillerde bu yapının olmamasını sevmediğini söyledi: MATLAB'da `do-while` yoktur, aynı kodu iki kez yazmak gerekir.

Soru: Fonksiyon kullansak olmaz mı?

Cevap: `repeat until` veya `do-while` temel bir döngü yapısıdır. Bir şeyi iki defa yapmak hocaya göre problemlidir, mümkünse tek yapmak gerekir.

Ders sonrası not: Hoca Python'da `do-while` olup olmadığını hatırlamadığını söyledi. Python'da büyük olasılıkla `do-while` yoktur, aynı iş `while True` ve `break` ile yazılır.

23. Laboratuvar 2: Seyrek Matrisin Sıkıştırılması

Kaydın son bölümünde aynı günün ikinci laboratuvarı vardır ve anlatımı asistan yapar.

23.1 Problem

Çok fazla sıfırı olan, yani seyrek bir A matrisi düşünelim. İstenen şey, yalnız sıfırdan farklı değerleri tutan bir sıkıştırılmış matris oluşturmaktır. Sıkıştırılmış matrisin her satırı bir elemanı anlatır:

Sütun	Tuttuğu bilgi
1.	Elemanın satırı
2.	Elemanın sütunu
3.	Elemanın değeri

Yani bu matris, A'daki sıfırdan farklı eleman sayısı kadar satıra ve 3 sütuna sahiptir. Slaytın başındaki örnekte indisler sıfırdan başlar: A'nın 0. satır 3. sütununda 1 varsa, sıkıştırılmış matrisin ilk satırı `0 3 1` olur. 1. satır 1. sütunda 2 varsa ikinci satırı `1 1 2` olur.

İkinci aşamada işlem tersine çevrilir. Sıkıştırılmış matris verilir, A geri oluşturulur. Slayttaki örnekte `3 2 1` için 3. satırın 2. sütununa 1, `1 4 3` için 1. satırın 4. sütununa 3 yazılır, aynı satırın son sütununda da -5 vardır. Geri kalan bütün gözler 0 olur.

23.2 Menü

Program bir menüyle yönetilir: 1 sıkıştırma, 2 geri oluşturma, 3 çıkış. Her işlem bittikten sonra menü yeniden gelir ve yeni işlem sorulur. Seçenek 1'de önce matrisin boyutları, sonra elemanları girilir ve sıkıştırılmış hali ekrana yazılır. Asistan, ilerideki veri yapıları ve algoritma analizi gibi derslerin ödevlerinde de menü hazırlamanın gerekeceğini, bunun iyi bir hazırlık olduğunu söyledi.

23.3 Asistanın cevapladığı sorular

Soru: Matris kare olmak zorunda mı?

Cevap: Hayır. Boyut $n \times m$ olabilir, boyutları kullanıcı girer.

Soru: Her satırda yalnız bir eleman mı var?

Cevap: Hayır. Bir satırda veya sütunda birden fazla sıfırdan farklı değer olabilir.

Soru: Matrisin değerleri koda yazılabilir mi, kullanıcıdan mı alınacak?

Cevap: Değerler ekrandan tek tek alınmalıdır. Testte az elemanlı bir örnek girilebilir.

Soru: Bütün elemanlar sıfırdan farklı ya da hepsi sıfır olursa?

Cevap: Hepsi sıfırdan farklıysa hepsi sıkıştırılmış matrise yazılır. Bu durumda sıkıştırmaktan çok genişletmiş olursunuz, çünkü yöntem doğası gereği çok sıfırlı matrisler içindir. Hepsi sıfırsa sıkıştırılmış matris boş olur.

Soru: Geri oluşturulan matris için boyut nasıl belirlenir?

Cevap: Büyük bir boyut tanımlanabilir. Örneğin sıkıştırılmış matristeki en büyük indis 4 ise en az 5×5 gerekir. İşi garantiye almak için 10×10 açılabilir. Hafızayı bol kullanmak burada sorun değildir. Ama büyük indisler sığmayacağı için bu boyut yeterince büyük seçilmelidir.

Soru: Kendi fonksiyonumuzu yazabilir miyiz?

Cevap: Evet. Hazır kütüphane fonksiyonları ise sınırlıdır. `printf`, `scanf` ve sistem çağrıları zaten kullanılır. Ama kendinizin kolayca yazabileceği işler, örneğin iki `for` ile yapılabilecek string karşılaştırması için `strcmp` gibi hazır fonksiyon kullanılmaması beklenir.

Kod yazım kuralı olarak da `for` içinde değişken tanımlanmaması, bütün tanımların derste gösterildiği gibi `main`'in hemen altında yapılması istendi.

Dersin Toparlaması

Bu derste iteratif algoritmaların analizi bitirildi. Yöntem her soruda aynıdır: Önce döngü değişkeninin nasıl ilerlediği bulunur, üst sınırda bilinmeyen varsa önce o hesaplanır, sonra içeriye toplanan şey yazılıp toplam içeriden dışarı çözülür. Doğrudan yazılamayan durumlarda, örneğin `if` içeren döngülerde, birkaç değerden desen çıkarılır ve `else` kolu "tüm durumlar eksi `if` durumu" olarak bulunur. Polinom toplamlarla üstel toplamlar farklıdır, geometrik seri ve $\sum i \cdot a^i$ formülleri de ispatlarıyla görüldü. `T`'nin değeri içeride ne toplandığına bağlıdır, karmaşıklık ise temel işlemin kaç kez yapıldığıdır.

İkinci kısımda rastgele sayılara giriş yapıldı. Çember üzerindeki desen örneği, tekrar sorununu ve tek düze dağılımı gösterdi. Bilgisayardaki üreteçlerin sözde rastgele olduğu, dizinin çekirdek değere bağlı olduğu ve `srand(time(NULL))` ile her çalıştırmada farklı bir çekirdek elde edildiği anlatıldı. Sonraki derste rastgele sayı üretimi kod üzerinden, zar, 52 kart, Gauss dağılımı, robot ve pi sayısı örnekleriyle sürecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.